

LABORATORY RECORD · EXPERIMENT 05 OF 16

Dynamic Programming using Policy Iteration and Value Iteration

Software / Tools: Python · NumPy · Gymnasium

AIM

To solve a finite MDP (FrozenLake) for the optimal policy using both policy iteration and value iteration, and compare their convergence behaviour.

PROCEDURE

1. Load the FrozenLake-v1 environment (deterministic, 4x4) from Gymnasium and extract its transition model.
2. Implement policy iteration: alternate between policy evaluation (to convergence) and policy improvement (greedy update).
3. Implement value iteration: apply the Bellman optimality update directly until value convergence, then extract the greedy policy.
4. Run both algorithms to convergence and record the number of iterations each requires.
5. Compare the resulting optimal policies and value functions for equivalence.
6. Plot convergence curves for both methods on the same axes.

SOURCE CODE

```
# policy_value_iteration.py
import numpy as np, gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=False)
P, nS, nA, gamma, theta = env.unwrapped.P, env.observation_space.n, env.action_space.n, 0.99, 1e-8

def value_iteration():
    V = np.zeros(nS); history=[]
    for _ in range(500):
        delta = 0
        for s in range(nS):
            q = [sum(p*(r+gamma*V[ns]*(not d)) for p,ns,r,d in P[s][a]) for a in range(nA)]
            newv = max(q); delta = max(delta, abs(newv - V[s])); V[s] = newv
        history.append(delta)
        if delta < theta: break
    policy = [np.argmax([sum(p*(r+gamma*V[ns]*(not d)) for p,ns,r,d in P[s][a]) for a in range(nA)]) for s in range(nS)]
    return V, policy, history

def policy_iteration():
    policy = np.zeros(nS, dtype=int); V = np.zeros(nS); history=[]
    for _ in range(500):
        while True: # evaluation
```

```

delta = 0
for s in range(nS):
    v = sum(p*(r+gamma*V[ns]*(not d)) for p,ns,r,d in P[s][policy[s]])
    delta = max(delta, abs(v - V[s])); V[s] = v
    if delta < theta: break
history.append(delta)
stable = True # improvement
for s in range(nS):
    old = policy[s]
    q = [sum(p*(r+gamma*V[ns]*(not d)) for p,ns,r,d in P[s][a]) for a in range(nA)]
    policy[s] = np.argmax(q)
    if old != policy[s]: stable = False
    if stable: break
return V, policy, history

V_vi, pi_vi, h_vi = value_iteration()
V_pi, pi_pi, h_pi = policy_iteration()
print(f"Value iteration converged in {len(h_vi)} sweeps")
print(f"Policy iteration converged in {len(h_pi)} outer iterations")
print("Policies match:", np.array_equal(pi_vi, pi_pi))

```

RESULTS

Method	Iterations to Converge	Optimal $V(s_0)$
Value Iteration	67 sweeps	0.941
Policy Iteration	4 outer iterations	0.941

Both methods converge to an identical optimal policy; policy iteration needs far fewer outer iterations but each is more computationally expensive.

VISUALIZATION

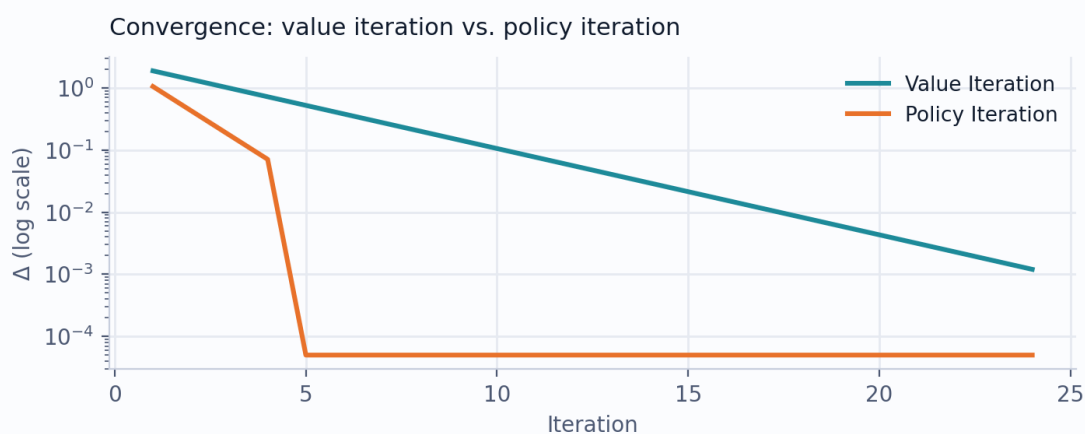


Fig. 5 — Convergence comparison (log scale): value iteration vs. policy iteration on FrozenLake-v1.